五点差分法求解 Poisson 方程中不同数值解法的比较分析

1 引言

泊松方程广泛应用于电磁学、流体动力学和热传导等领域。在许多实际应用中，直接求解泊松方程的解析解十分复杂。因此，开发高效的数值求解方法对于这类问题的研究至关重要。本报告旨在通过 python 数值实验，比较和分析不同数值求解方法在求解定界泊松方程时的性能。特别地，我们关注于离散正弦变换（DST）方法的实现和效率，考察了其在不同网格划分下的求解精度和运行时间，并与其他数值方法如 Jacobi 迭代、Gauss-Seidel 迭代和 LU 分解方法进行了比较。

2 问题描述

对于一个二维矩形域上的满足齐次 Dirichlet 边界条件的 Poisson 方程，其形式为：

$$\begin{cases}\Delta u(x,y)=f(x,y),(x,y)\in\Omega\\ u(x,y)=0,(x,y)\in\partial\Omega\end{cases},\Omega=(0,a)\times(0,b)$$

我们需要使用一系列数值解法来求解上述 Poisson 方程，并对比分析不同解法的运算效率。

在本次数值实验中，我所选择的具体算例为：

$$\begin{cases}\Delta u(x,y)=-2\pi^2\sin\pi x\sin\pi y,(x,y)\in\Omega\\ u(x,y)=0,(x,y)\in\partial\Omega\end{cases},\Omega=(0,1)\times(0,1)$$

这个方程有解析解$u=\sin\pi x\sin\pi y$，能够让我们更加方便地分析数值结果的准确性。

3 数值方法

3.1 五点差分

将矩形区域在x方向划分为$I+1$等份，$x_i=ih$，y方向划分为$J+1$等份，$y_j=jh$。将方程的信息离散化（五点差分）：

$$\Delta_h u_{ij}:=\frac{u_{i+1,j}-2u_{ij}+u_{i-1,j}}{h^2}+\frac{u_{i,j+1}-2u_{ij}+u_{i,j-1}}{k^2}$$

即：

$$\begin{cases} \Delta_h u_{ij} = f_{ij}, 1 \le i \le I, 1 \le j \le J \\ u_{ij} = \alpha_{ij}, i \in \{0, I+1\}或j \in \{0, J+1\} \end{cases}$$

这种划分求解方式具有稳定性和唯一性，具有二阶精度$O(h^2)$。

3.2 迭代法求解

3.2.1 Jacobi 迭代法

$$\begin{cases} u_{ij}^{new} = \frac{1}{4}\left(u_{i+1,j}^{old} + u_{i-1,j}^{old} + u_{i,j+1}^{old} + u_{i,j-1}^{old}\right) - \frac{1}{4}h^2 f_{ij} \\ u_{ij}^{new} = u_{ij}^{old} = \alpha_{ij} \end{cases}$$

在用 python 实现 Jacobi 迭代法时，我注意到除了使用两层 for 循环遍历矩阵U_{old}中的每一个元素并利用其周围的值更新U_{new}外，还可以使用卷积的方式来完成计算。

对于x轴划分步长为h，y轴划分步长为k的情况，卷积核K可以表示为：

$$K = \begin{bmatrix} 0 & \frac{1}{k^2} & 0 \\ \frac{1}{h^2} & 0 & \frac{1}{h^2} \\ 0 & \frac{1}{k^2} & 0 \end{bmatrix}$$

那么矩阵U_{new}和U_{old}可以表示为：

$$U_{new} = (U_{old} * K - F) \cdot \frac{1}{\frac{2}{h^2} + \frac{2}{k^2}}$$

在卷积时为了保持U_{new}的形状不变并且边界条件被考虑在内，U_{new}需要在其边界进行补零操作。

scipy 提供的卷积方法 scipy.signal.convolve2d 是一个已经高度优化的函数，对卷积计算有明显的加速。而对于后续的 Gauss-Seidel 迭代法和 SOR 迭代法由于需要按照一定顺序进行计算，我无法用卷积的形式去表示。这可能导致利用卷积加速的 Jacobi 迭代法的迭代次数高于 Gauss-Seidel 迭代法和 SOR 迭代法，但总运算速度却快得多。为了更加全面的分析不同迭代法的差距，我编写了两个实践 Jacobi 迭代法的方法：一个是 method_jacobi_iteration，用 for 循环实现；另一个是 method_jacobi_iteration_convolve，用 scipy.signal.convolve2d 实现。

3.2.2 Gauss-Seidel 迭代法

Gauss-Seidel 迭代法是 Jacobi 迭代法的一个改进版本，其迭代公式考虑了最新的迭代值，这样可以加快收敛速度。在 Gauss-Seidel 方法中，每次迭代时使用已经计算出的新值更新未知数，迭代公式为：

$$\begin{cases} u_{ij}^{new} = \frac{1}{4}\left(u_{i+1,j}^{new} + u_{i-1,j}^{new} + u_{i,j+1}^{old} + u_{i,j-1}^{old}\right) - \frac{1}{4}h^2 f_{ij} \\ u_{ij}^{new} = u_{ij}^{old} = \alpha_{ij} \end{cases}$$

与 Jacobi 方法相比，Gauss-Seidel 方法在迭代过程中，一旦计算出一个新的分量，就立即将其用于计算后续的分量。

3.2.3 SOR 迭代法

SOR 方法是 Gauss-Seidel 方法的推广，引入了一个松弛因子ω来加速收敛。通过对 Gauss-Seidel 迭代公式的每次迭代结果进行加权平均，可以得到 SOR 迭代公式：

$$\begin{cases} \tilde{u}_{ij}^{new} = \frac{1}{4}\left(\tilde{u}_{i+1,j}^{new} + \tilde{u}_{i-1,j}^{new} + u_{i,j+1}^{old} + u_{i,j-1}^{old}\right) - \frac{1}{4}h^2 f_{ij} \\ \tilde{u}_{ij}^{new} = u_{ij}^{old} = \alpha_{ij} \end{cases}$$

$$u_{ij}^{new} = \omega\tilde{u}_{ij}^{new} + (1-\omega)u_{ij}^{old}$$

当$\omega = 1$时，SOR 方法退化为 Gauss-Seidel 方法。选择合适的ω可以显著提高迭代的收敛速度。本次数值实验中我选取的ω为 1.5。

3.3 解线性方程组

将五点差分格式写为矩阵的形式：

矩阵形式：

$$U = \begin{bmatrix} u_{11} & u_{12} & \cdots & u_{1J} \\ u_{21} & u_{22} & \cdots & u_{2J} \\ \vdots & \vdots & \ddots & \vdots \\ u_{I1} & u_{I2} & \cdots & u_{IJ} \end{bmatrix}, F = -\begin{bmatrix} f_{11} & f_{12} & \cdots & f_{1J} \\ f_{21} & f_{22} & \cdots & f_{2J} \\ \vdots & \vdots & \ddots & \vdots \\ f_{I1} & f_{I2} & \cdots & f_{IJ} \end{bmatrix}$$

$$A = \frac{1}{h^2}\begin{bmatrix} 2 & -1 & & \\ -1 & 2 & \ddots & \\ & \ddots & \ddots & -1 \\ & & -1 & 2 \end{bmatrix}_{I\times I}, B = \frac{1}{k^2}\begin{bmatrix} 2 & -1 & & \\ -1 & 2 & \ddots & \\ & \ddots & \ddots & -1 \\ & & -1 & 2 \end{bmatrix}_{I\times I}$$

$$AU + UB = F$$

将U和F按照列拉直，得到一个线性方程组：

$$(I \otimes A + B \otimes I)\vec{U} = \vec{F}$$

这个线性方程组可以使用 LU 分解来求解。LU 分解可以由 scipy.linalg.lu_factor 来实现，分解后的求解可以用 scipy.linalg.lu_solve 来实现。

3.4 离散正弦傅里叶变换

利用线性代数的相关知识可以推导出U的表达式：

$$U=\sum_{i=1}^{I}\sum_{j=1}^{J}\frac{4\boldsymbol{u}_i^T F\boldsymbol{v}_j}{(\lambda_i+\mu_j)(I+1)(J+1)}\boldsymbol{u}_i\boldsymbol{v}_j^T$$

其中$\boldsymbol{u}_i$和$\boldsymbol{v}_i$是矩阵A,B的特征向量，λ_i,μ_i是矩阵A,B的特征值。

将矩阵A的特征向量按照列排成一个$I\times I$矩阵P，矩阵B的特征向量按照列排成一个$J\times J$矩阵Q。P的第(m,n)个元素为$\sin\frac{mn\pi}{I+1}$，Q的第(m,n)个元素为$\sin\frac{mn\pi}{J+1}$。那么U可以表示为：

$$V=PFQ$$

$$w_{ij}=\frac{4v_{ij}}{(\lambda_i+\mu_j)(I+1)(J+1)}$$

$$U=PWQ$$

在完成上述计算时，最重要的是如何快速计算$V=PFQ$与$U=PWQ$两个大型矩阵运算。考虑到P,Q两个矩阵的每一项的特点，可以用离散正弦变换来加速这一矩阵乘法计算，即对F或Q进行一次按行的离散正弦变换和按列的离散正弦变换。

scipy.fft 中提供了离散正弦变换的函数 dst。该函数提供了四种类型的离散正弦变换，通过 type 参数指定，默认是第二类离散正弦变换（type=2）。符合本次矩阵乘法的是 type=1，并且再此基础上还需要除以二。并且该函数还接受参数 axis，指定是按照行进行离散正弦变换还是按照列进行离散正弦变换。具体实现利用 DST 加速矩阵运算的代码为：
dst_transform = lambda mat: dst(dst(mat, type=1, axis=1),type=1, axis=0)/4

4 实验方法

本次数值实验所求解的方程是定义在$\Omega=(0,x_{max})\times(0,y_{max})$上的满足齐次 Dirichlet 边界条件的 Poisson 方程。作业所要求的问题 1 和问题 3 均已在第 3 节所说的所有方法中实现。而问题 2 我会在后续解决。

具体代码逻辑如下：

1. 主类 **NumericalSolutionForPossionEquation**：
   a) 初始化方法(**__init__**)：接收格点函数、x 轴和 y 轴的最大值、以及格点数（n），计算格点间隔和 x、y 坐标点。
   b) **_generate_matrix_AB** 方法：生成泊松方程离散化后的系数矩阵 A 和 B。
   c) **_generate_eigenvalues_AB** 方法：计算矩阵 A 和 B 的特征值。
   d) 数值解法
      i. 离散傅里叶变换(**method_dst**)

ii. LU 分解(**method_LUdecomposition**)

iii. Jacobi 迭代(**method_jacobi_iteration** 和 **method_jacobi_iteration_convolve**)

iv. 高斯-塞德尔迭代(**method_gauss_seidel_iteration**)

v. SOR 迭代(**method_sor_iteration**)。

e) **check_self** 方法：验证数值解的自洽性。

f) **check_analytical_solution** 方法：与解析解进行比较。

g) **visualize** 方法：可视化数值解。

h) **analyze_solution_time_error** 方法：分析不同求解方法的时间消耗和误差。

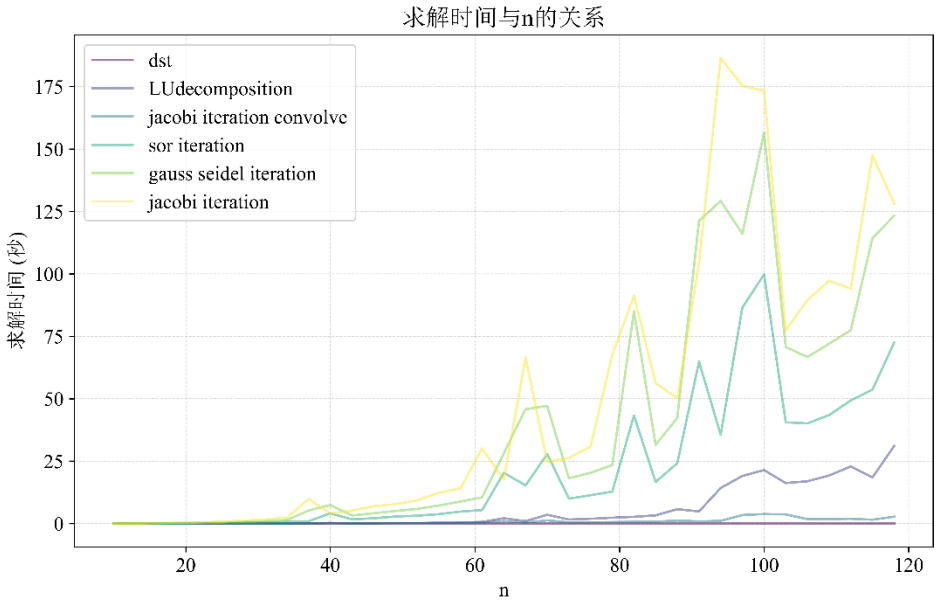
2. 主函数 **time_error_analysis**：对不同的格点数量（n），通过多种数值解法求解泊松方程，并分析求解时间和误差。

3. 子类 **NonhomogeneousPoissonEquation**：推广至非齐次条件

4. 实例：应用主函数于第 2 节中所举的例子，格点划分数n从 20 增加至 120。

5 实验结果

5.1 求解时间



由图中可以看出，随着划分格点数n的增加，求解时间大体呈递增的趋势。而运算速度的排序为：

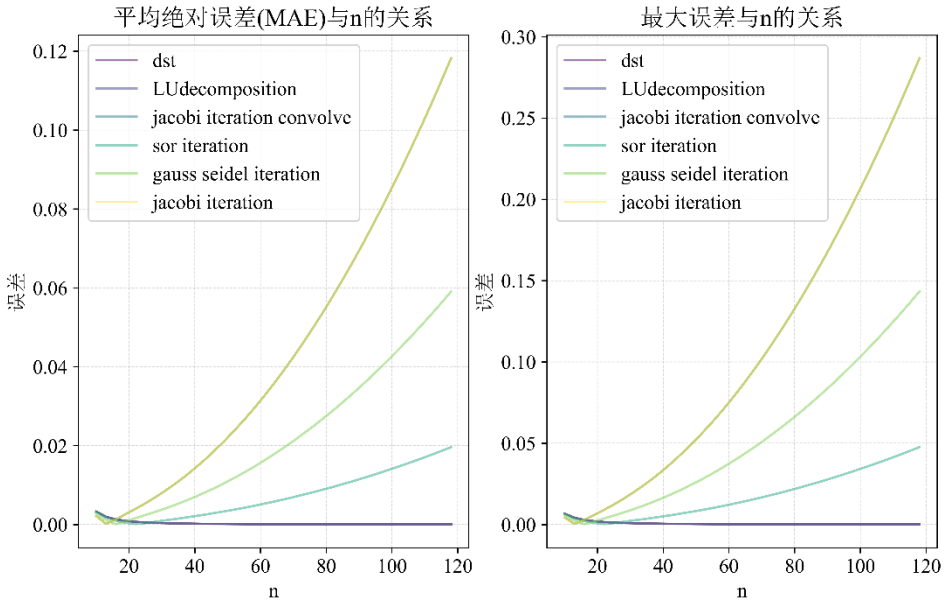
DST>Jacobi 迭代（卷积加速）>LU 分解>SOR 迭代>Gauss-Seidel 迭代>Jacobi 迭代（无卷积加速）

具体的求解时间列表如下：（单位：秒）

方法 / n	DST	LU	Jacobi-卷积	SOR	Gauss-Seidel	Jacobi
20	0.000	0.010	0.011	0.120	0.212	0.334
40	0.000	0.060	0.093	1.118	2.046	3.604
60	0.001	0.470	0.294	4.843	12.153	21.049
80	0.001	3.636	0.737	19.035	28.867	49.752
100	0.004	9.781	1.948	43.808	71.072	90.266

与没有卷积加速的 Jacobi 迭代法相比，DST 方法快了约三万倍。

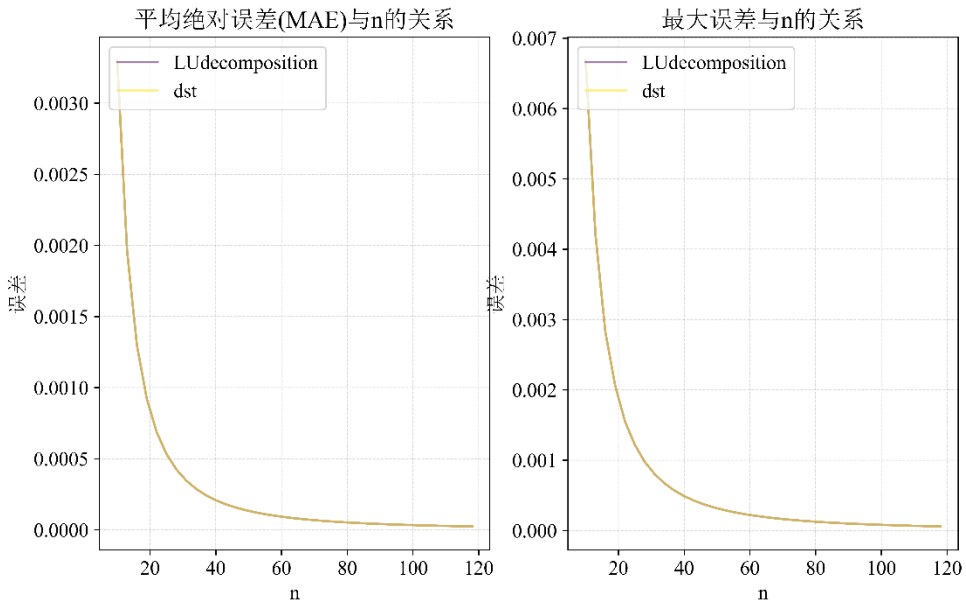
5.2 求解精度



图中 Jacobi-卷积加速和 Jacobi 曲线完全重合，DST 和 LU 分解法曲线完全重合。

注意到随着n的增加迭代方法产生的误差反而更大了。这可能和迭代方法的相关参数有关。在迭代时我们需要判断何时终止迭代，我的代码中判断逻辑为当$U_{new} - U_{old}$的绝对值的最大元素小于容忍项$tol = 1.0 \times 10^{-4}$时终止迭代。在进行实验时，容忍项tol的值是一个定值，没有随n一同改变。然而实际上为了达到相同的精度，提高n的值的同时也需要减小tol的值。所以图中的误差随着n的增大反而减小的现象可能是正常情况，不能据此说明迭代法不收敛。同时由于tol项的存在，根据图像比较不同迭代法的精度可能没有意义。

在图中 DST 和 LU 分解方法的变化情况不明显。对于这两种方法绘图如下：



二者都是求解同一个矩阵方程，只是计算方法不同，故二者的误差完全一致。并且随着n的增加，截断误差单调递减。这与理论上五点差分法的误差为$O(h^2)$相符合。

具体计算误差列表如下：

平均绝对误差：

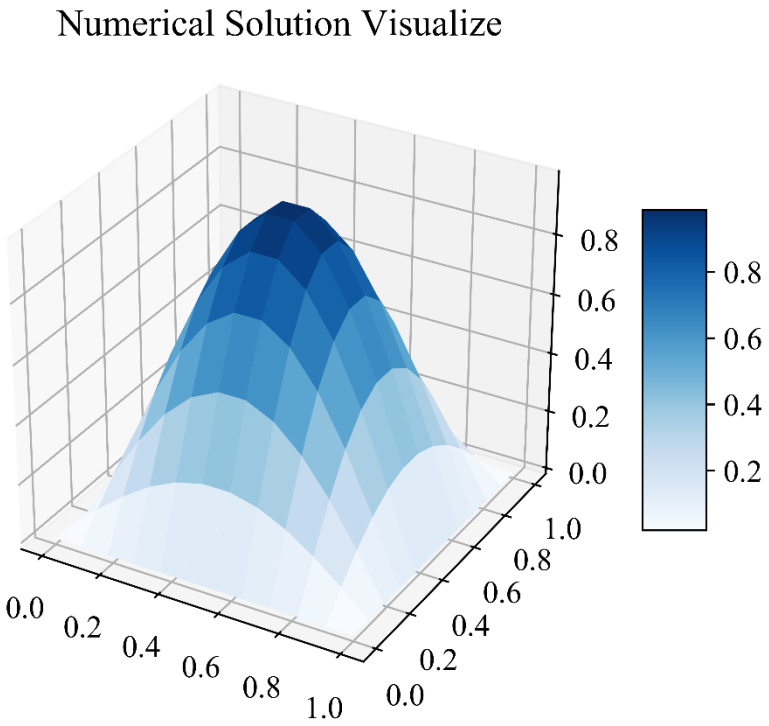
方法 / n	DST	LU	Jacobi-卷积	SOR	Gauss-Seidel	Jacobi
20	0.00083	0.00083	0.00311	0.00028	0.00109	0.00311
40	0.00021	0.00021	0.01425	0.00213	0.00697	0.01425
60	0.00009	0.00009	0.03146	0.00510	0.01565	0.03146
80	0.00005	0.00005	0.05514	0.00906	0.02753	0.05514
100	0.00003	0.00003	0.08539	0.01414	0.04265	0.08539

最大误差：

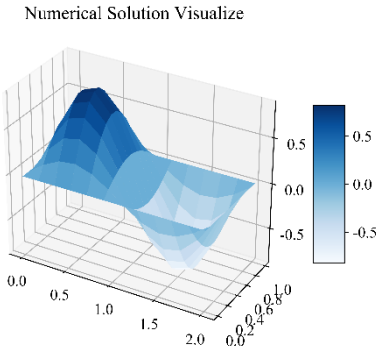
方法 / n	DST	LU	Jacobi-卷积	SOR	Gauss-Seidel	Jacobi
20	0.0019	0.0019	0.0069	0.0007	0.0025	0.0069
40	0.0005	0.0005	0.0334	0.0051	0.0164	0.0334
60	0.0002	0.0002	0.0751	0.0122	0.0374	0.0751
80	0.0001	0.0001	0.1327	0.0218	0.0663	0.1327
100	0.0001	0.0001	0.2065	0.0342	0.1032	0.2065

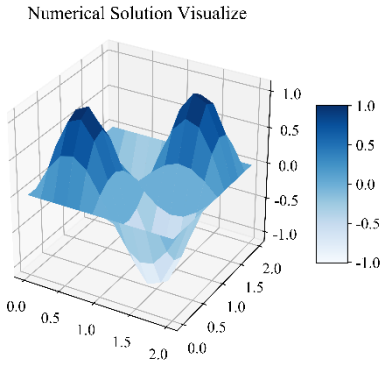
5.3 求解结果可视化

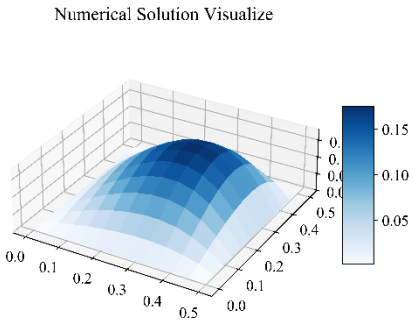
利用 DST 方法求解结果如下：

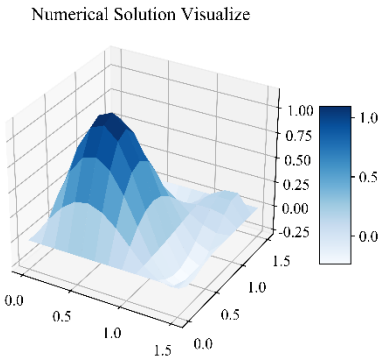


将定义域由$[0,1]\times[0,1]$更改为$[0,a]\times[0,b]$：









6 非齐次的方程求解方法

对于非齐次 Dirichlet 边界条件的 Poisson 方程，其格式如下：

$$\begin{cases}\Delta u(x,y)=f(x,y),(x,y)\in\Omega\\ u(x,y)=\alpha(x,y),(x,y)\in\partial\Omega\end{cases}$$

6.1 迭代法

对于迭代法，以卷积加速的 Jacobi 迭代法为例，如果需要修改使得其能够适应，则需要修改卷积时对边界值的填充。齐次方程求解时边界值均填充的是 0，而非齐次方程则需要根据$\alpha(x,y)$的值来填充。

6.2 矩阵方程

而对于矩阵方程，我们则需要修改F矩阵的中$f_{1j},f_{i1},f_{nj},f_{in}$的值使其适配非齐次 Dirichlet 边界条件。具体修正方案如下：

齐次项	非齐次修正
$-\boldsymbol{f_{1j}}=\dfrac{\boldsymbol{2u_{1j}-u_{2j}}}{\boldsymbol{h^2}}+\dfrac{\boldsymbol{2u_{j1}-u_{(j-1)1}-u_{(j+1)1}}}{\boldsymbol{k^2}}$	$-f_{1j}^{*}=-f_{1j}-\dfrac{u_{0j}}{h^2}$
$-\boldsymbol{f_{i1}}=\dfrac{\boldsymbol{2u_{i1}-u_{(i-1)1}-u_{(i+1)1}}}{\boldsymbol{h^2}}+\dfrac{\boldsymbol{2u_{i1}-u_{i2}}}{\boldsymbol{k^2}}$	$-f_{1j}^{*}=-f_{1j}-\dfrac{u_{i0}}{k^2}$
$-\boldsymbol{f_{nj}}=\dfrac{\boldsymbol{2u_{nj}-u_{(n-1)j}}}{\boldsymbol{h^2}}+\dfrac{\boldsymbol{2u_{jn}-u_{(j-1)n}-u_{(j+1)n}}}{\boldsymbol{k^2}}$	$-f_{1j}^{*}=-f_{1j}-\dfrac{u_{(n+1)j}}{h^2}$
$-\boldsymbol{f_{in}}=\dfrac{\boldsymbol{2u_{in}-u_{(i-1)n}-u_{(i+1)n}}}{\boldsymbol{h^2}}+\dfrac{\boldsymbol{2u_{in}-u_{i(n-1)}}}{\boldsymbol{k^2}}$	$-f_{1j}^{*}=-f_{1j}-\dfrac{u_{i(n+1)}}{k^2}$

注意四个角的值要修正两次。同时代码中也需要注意，矩阵方程中的F矩阵需要更改，而迭代法中的F矩阵不需要更改。

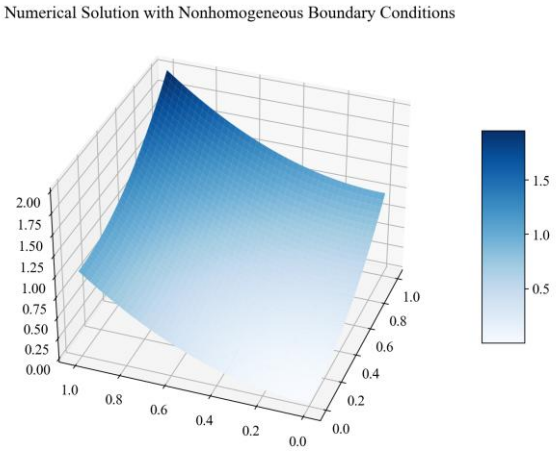
6.3 求解结果

示例方程：

$$\begin{cases}\Delta u=4,x\in\Omega\\ u=x^2+y^2,x\in\partial\Omega\end{cases},\Omega=(0,1)\times(0,1)$$

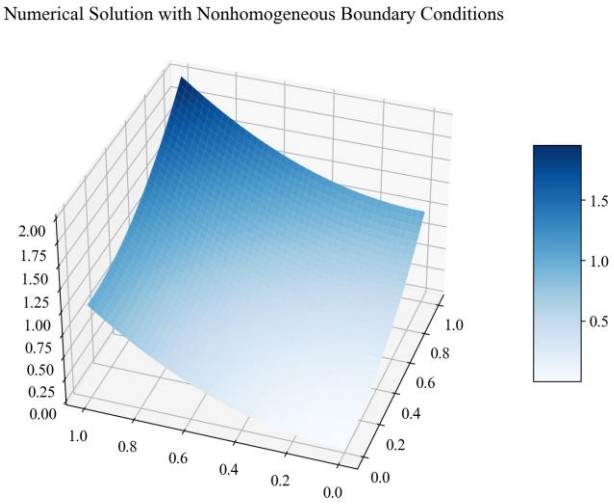
这个方程的解析解为$u=x^2+y^2$。

卷积加速的 Jacobi 迭代法求解结果（$n=40$）：



与解析解的偏差：最大偏差为 0.03394，平均绝对误差为 0.01445

DST 求解的结果：



与解析解的偏差：最大偏差1.332×10^{-15}，平均偏差2.249×10^{-16}。

以上结果表明代码能够顺利地求解非齐次边界条件下的微分方程。

7 总结

本次 python 数值实验综合比较和分析了五点差分法在求解矩形域上 Poisson 方程时不同数值解法的性能。实验涵盖了离散正弦变换（DST）、Jacobi 迭代、Gauss-Seidel 迭代、LU 分解、SOR 迭代这几种方法，在不同网格划分下评估了它们的求解精度和运行时间。结果表明，DST 方法在运行速度上显著优于其他方法。在求解精度方面，DST 和 LU 分解方法表现出了较高的准确性。同时，求解代码能够将迭代方法和矩阵方程方法拓充至$[0, a] \times [0, b]$与非齐次边界条件的情况。

8 附录

```python
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import time
import os
from scipy.fft import dst
from scipy.linalg import lu_factor, lu_solve
from mpl_toolkits.mplot3d import Axes3D
from scipy.signal import convolve2d

from matplotlib import font_manager
from matplotlib import rcParams
font_path = "times+simsun.ttf"
font_manager.fontManager.addfont(font_path)
prop = font_manager.FontProperties(fname=font_path)
rcParams['font.family'] = 'sans-serif'
rcParams['font.sans-serif'] = prop.get_name()
rcParams['font.size'] = 13
rcParams['axes.unicode_minus'] = False

class NumericalSolutionForPossionEquation:

    def __init__(
        self,
        lattice_function:callable, # 格点函数
        x_max:float=1.0, # x 轴范围：[0,x_max]
        y_max:float=1.0, # y 轴范围：[0,y_max]
        n:int=10, # n+1 等分 x 轴与 y 轴
        ):
        self.x_max = x_max
        self.y_max = y_max
        self.lattice_function = lattice_function
        self.n = n

        self.x_step = x_max/(n+1)
        self.y_step = y_max/(n+1)

        self.x_points = np.linspace(0, x_max, n+2)[1:-1]
        self.y_points = np.linspace(0, y_max, n+2)[1:-1]
        self.matrix_F = lattice_function(self.x_points[:, None], self.y_points)

```

```
42.
43.     def _generate_matrix_AB(self):
44.         mat = np.zeros((self.n, self.n))
45.         np.fill_diagonal(mat, 2)
46.         np.fill_diagonal(mat[:-1, 1:], -1)
47.         np.fill_diagonal(mat[1:, :-1], -1)
48.         return mat
49.
50.     def _generate_eigenvalues_AB(self, step):
51.         return 4/step**2*np.array([np.sin(i*np.pi*step/2)**2 for i in range(1,
    self.n+1)])
52.
53.     def method_dst(self):
54.         """ 离散傅里叶正弦变换求解 """
55.         dst_transform = lambda mat: dst(dst(mat, type=1, axis=1),type=1, axis=0
    )/4
56.         eigenvalues_A = self._generate_eigenvalues_AB(self.x_step)
57.         eigenvalues_B = self._generate_eigenvalues_AB(self.y_step)
58.         V = dst_transform(-self.matrix_F) # 第一个大型矩阵运算：PFQ
59.         W = V/(eigenvalues_A[:, None] + eigenvalues_B)*4*self.x_step*self.y_ste
    p # 利用 numpy 的广播机制计算 W
60.         U = dst_transform(W) # 第二个大型矩阵运算：PWQ
61.         return U
62.
63.     def method_LUdecomposition(self):
64.         """ 将矩阵 U 拉直，转化为线性方程组使用 LU 分解求解 """
65.         matrix_A = 1/self.x_step**2*self._generate_matrix_AB()
66.         matrix_B = 1/self.y_step**2*self._generate_matrix_AB()
67.         I = np.eye(self.n)
68.         coefficient_matrix = np.kron(I, matrix_A) + np.kron(matrix_B, I)
69.         resultant_vector = -self.matrix_F.reshape(self.n*self.n, 1)
70.
71.         lu, piv = lu_factor(coefficient_matrix)
72.         x = lu_solve((lu, piv), resultant_vector)
73.         U = x.reshape(self.n, self.n)
74.         return U
75.
76.     def method_jacobi_iteration(self, max_iter=20000, tol=1e-4):
77.         """ 利用 Jacobi 迭代法求解 """
78.         iter_count = 0
79.         U_new = np.zeros((self.n, self.n))
80.         U_old = np.zeros((self.n, self.n))
81.         while iter_count < max_iter:
```

```
            U_old_pad = np.pad(U_old, pad_width=1, mode='constant', constant_values=0)
            for i in range(self.n):
                for j in range(self.n):
                    i_pad = i+1; j_pad = j+1
                    value_1 = (U_old_pad[i_pad+1, j_pad]+U_old_pad[i_pad-1, j_pad])/self.x_step**2
                    value_2 = (U_old_pad[i_pad, j_pad+1]+U_old_pad[i_pad, j_pad-1])/self.y_step**2
                    value_3 = self.matrix_F[i, j]
                    value_4 = (2/self.x_step**2+2/self.y_step**2)
                    U_new[i, j] = (value_1 + value_2 - value_3)/value_4

            diff = np.max(np.abs(U_old - U_new))
            if diff < tol:
                break
            iter_count += 1
            U_old = U_new.copy()
        else:
            print(f"注意：本方法(method_jacobi_iteration)已经超过最大迭代数{max_iter}，结果可能不收敛。")
        return U_new

    def method_jacobi_iteration_convolve(self, max_iter=20000, tol=1e-4):
        """ 利用 Jacobi 迭代法求解，迭代过程用 scipy 提供的卷积函数加速 """
        factor = 1 / (2/self.x_step**2 + 2/self.y_step**2)
        kernel = np.array([
            [0, 1/self.y_step**2, 0],
            [1/self.x_step**2, 0, 1/self.x_step**2],
            [0, 1/self.y_step**2, 0],
            ]) * factor
        iter_count = 0

        U_new = np.zeros((self.n,self.n))
        U_old = np.zeros((self.n,self.n))

        while iter_count < max_iter:
            U_new = convolve2d(U_old, kernel, mode='same', boundary='fill', fillvalue=0) - self.matrix_F * factor
            diff = np.max(np.abs(U_old - U_new))
            if diff < tol:
                break
            U_old = U_new.copy()
            iter_count += 1
```

```
121.             else:
122.                 print(f"注意：本方法(method_jacobi_iteration_convolve)已经超过最大迭代数{max_iter}，结果可能不收敛。")
123.             return U_new
124.
125.         def method_gauss_seidel_iteration(self, max_iter=20000, tol=1e-4):
126.             """ 利用 Gauss Seidel 迭代法求解 """
127.             iter_count = 0
128.             U_new = np.zeros((self.n, self.n))
129.             U_old = np.zeros((self.n, self.n))
130.
131.             while iter_count < max_iter:
132.                 for i in range(self.n):
133.                     for j in range(self.n):
134.                         u_left = U_new[i-1, j] if i > 0 else 0
135.                         u_right = U_old[i+1, j] if i < self.n-1 else 0
136.                         u_down = U_new[i, j-1] if j > 0 else 0
137.                         u_up = U_old[i, j+1] if j < self.n-1 else 0
138.
139.                         value_1 = (u_left + u_right) / self.x_step**2
140.                         value_2 = (u_down + u_up) / self.y_step**2
141.                         value_3 = self.matrix_F[i, j]
142.                         value_4 = (2 / self.x_step**2 + 2 / self.y_step**2)
143.
144.                         U_new[i, j] = (value_1 + value_2 - value_3) / value_4
145.
146.                 diff = np.max(np.abs(U_new - U_old))
147.                 if diff < tol:
148.                     break
149.
150.                 U_old = U_new.copy()
151.                 iter_count += 1
152.             else:
153.                 print(f"注意：本方法(method_gauss_seidel_iteration)已经超过最大迭代数{max_iter}，结果可能不收敛。")
154.
155.             return U_new
156.
157.         def method_sor_iteration(self, omega=1.5, max_iter=20000, tol=1e-4):
158.             """ 利用 SOR 迭代法求解 """
159.             iter_count = 0
160.             U_new = np.zeros((self.n, self.n))
161.             U_old = U_new.copy()
162.
```

```
163.           while iter_count < max_iter:
164.               U_old = U_new.copy()
165.               for i in range(self.n):
166.                   for j in range(self.n):
167.                       u_left = U_new[i-1, j] if i > 0 else 0
168.                       u_right = U_old[i+1, j] if i < self.n-1 else 0
169.                       u_down = U_new[i, j-1] if j > 0 else 0
170.                       u_up = U_old[i, j+1] if j < self.n-1 else 0
171.
172.                       value_1 = (u_left + u_right) / self.x_step**2
173.                       value_2 = (u_down + u_up) / self.y_step**2
174.                       value_3 = self.matrix_F[i, j]
175.                       value_4 = (2 / self.x_step**2 + 2 / self.y_step**2)
176.
177.                       U_new[i, j] += omega * (((value_1 + value_2 - value_3) /
    value_4) - U_new[i, j])
178.
179.               diff = np.max(np.abs(U_new - U_old))
180.               if diff < tol:
181.                   break
182.
183.               iter_count += 1
184.
185.           else:
186.               print(f"注意：本方法(method_sor_iteration)已经超过最大迭代数
    {max_iter}，结果可能不收敛。")
187.
188.           return U_new
189.
190.       def check_self(self, U):
191.           tol = np.mean(np.abs(self.matrix_F))/100
192.           kernel = np.array([
193.               [0, 1 / self.y_step**2, 0],
194.               [1 / self.x_step**2, -
    2 * (1 / self.x_step**2 + 1 / self.y_step**2), 1 / self.x_step**2],
195.               [0, 1 / self.y_step**2, 0],
196.               ])
197.
198.           F_estimated = convolve2d(U, kernel, mode='same', boundary='fill', fi
    llvalue=0)
199.           diff = np.abs(F_estimated - self.matrix_F)
200.           max_diff = np.max(diff)
201.           mean_diff = np.mean(diff)
202.
```

```
203.             return max_diff, mean_diff
204.
205.         def check_analytical_solution(self, U, analytical_solution):
206.             X, Y = np.meshgrid(self.x_points, self.y_points, indexing='ij')
207.             U_analytical = analytical_solution(X, Y)
208.             max_diff = np.max(np.abs(U - U_analytical))
209.             mae = np.mean(np.abs(U - U_analytical))
210.
211.             return max_diff, mae
212.
213.         def visualize(self, U):
214.             X, Y = np.meshgrid([0]+list(self.x_points)+[self.x_max], [0]+list(se
    lf.y_points)+[self.y_max])
215.             U_pad = np.pad(U, pad_width=1, mode='constant', constant_values=0)
216.             fig = plt.figure()
217.             ax = fig.add_subplot(111, projection='3d')
218.             surf = ax.plot_surface(X, Y, U_pad.T, cmap='Blues')
219.             fig.colorbar(surf, shrink=0.5, aspect=5)
220.
221.             ax.set_title('Numerical Solution Visualize')
222.             ax.set_aspect('equal')
223.             plt.savefig("result.png", dpi=800)
224.             plt.show()
225.
226.         def analyze_solution_time_error(self, analytical_solution=None):
227.             methods = [
228.                 'method_dst',
229.                 'method_LUdecomposition',
230.                 'method_jacobi_iteration_convolve',
231.                 'method_sor_iteration',
232.                 'method_gauss_seidel_iteration',
233.                 'method_jacobi_iteration',
234.                 ]
235.             times = []
236.             max_error = []
237.             mean_absolute_error = []
238.             print(f"初始化完成，准备开始求解微分方程。{self.n=}")
239.             for method in methods:
240.                 start_time = time.time()
241.                 U = getattr(self, method)()
242.                 end_time = time.time()
243.                 if analytical_solution:
244.                     max_diff, mae = self.check_analytical_solution(U, analytical
    _solution)
```

```
                    max_error.append(max_diff)
                    mean_absolute_error.append(mae)
                times.append(end_time - start_time)
                print(f"\t{method}\t{end_time-start_time:.6f}秒。")

            solution_info = pd.DataFrame(times, index=[i.replace("method_","") f
    or i in methods], columns=['Execution Time (s)'])
            if analytical_solution:
                solution_info["MAE"] = mean_absolute_error
                solution_info["Max Error"] = max_error
            return solution_info


    def time_error_analysis(
        lattice_function,
        x_max, y_max,
        analytical_solution,
        n_values, method_list=[
                'method_dst',
                'method_LUdecomposition',
                'method_jacobi_iteration_convolve',
                'method_sor_iteration',
                'method_gauss_seidel_iteration',
                'method_jacobi_iteration',
                ]):
        output_folder = "Solution-Analysis"
        os.makedirs(output_folder, exist_ok=True)
        # 初始化存储结果的字典
        times = {method: [] for method in method_list}
        maes = {method: [] for method in method_list}
        max_errors = {method: [] for method in method_list}

        # 遍历不同的 n 值
        for n in n_values:
            print(n)
            solution = NumericalSolutionForPossionEquation(lattice_function, x_m
    ax, y_max, n)
            for method in method_list:
                start_time = time.time()
                U = getattr(solution, method)()
                end_time = time.time()

                max_diff, mae = solution.check_analytical_solution(U, analytical
    _solution)
```

```
                times[method].append(end_time - start_time)
                maes[method].append(mae)
                max_errors[method].append(max_diff)

        # 绘制求解时间与 n 的关系图像
        plt.figure(figsize=(10, 6))
        colors = plt.cm.viridis(np.linspace(0, 1, len(method_list)))
        for i, method in enumerate(method_list):
            plt.plot(n_values, times[method], label=method.replace("method_", ""
    ).replace("_", " "), color=colors[i], alpha=0.5)
        plt.xlabel('n')
        plt.ylabel('求解时间 (秒)')
        plt.title('求解时间与 n 的关系')
        plt.legend(loc='upper left')
        plt.grid(True, which='both', linestyle='--', linewidth=0.5, alpha=0.5)
        plt.savefig(os.path.join(output_folder, "time_vs_n.png"), dpi=800)
        plt.close()

        # 绘制 MAE 和最大误差与 n 的关系图像
        fig, axs = plt.subplots(1, 2, figsize=(10, 6))
        for i, method in enumerate(method_list):
            axs[0].plot(n_values, maes[method], label=method.replace("method_",
    "").replace("_", " "), color=colors[i], alpha=0.5)
            axs[1].plot(n_values, max_errors[method], label=method.replace("meth
    od_", "").replace("_", " "), color=colors[i], alpha=0.5)

        axs[0].set_title('平均绝对误差(MAE)与 n 的关系')
        axs[1].set_title('最大误差与 n 的关系')
        for ax in axs:
            ax.set_xlabel('n')
            ax.set_ylabel('误差')
            ax.legend(loc='upper left')
            ax.grid(True, which='both', linestyle='--
    ', linewidth=0.5, alpha=0.5)
        plt.savefig(os.path.join(output_folder, "errors_vs_n.png"), dpi=800)
        plt.show()
        plt.close()

        # 将数据保存到 Excel 文件中
        with pd.ExcelWriter(os.path.join(output_folder, "analysis_results.xlsx")
    ) as writer:
            pd.DataFrame(times, index=n_values).to_excel(writer, sheet_name='Tim
    es')
```

```
323.             pd.DataFrame(maes, index=n_values).to_excel(writer, sheet_name='MAEs
    ')
324.             pd.DataFrame(max_errors, index=n_values).to_excel(writer, sheet_name
    ='Max Errors')
325.
326.
327.   class NonhomogeneousPoissonEquation(NumericalSolutionForPossionEquation):
328.       def __init__(
329.           self, lattice_function: callable,
330.           boundary_left: callable=lambda x: x*0, boundary_right: callable=lamb
    da x: x*0,
331.           boundary_top: callable=lambda x: x*0, boundary_bottom: callable=lamb
    da x: x*0,
332.           *args, **kwargs
333.       ):
334.           super().__init__(lattice_function, *args, **kwargs)
335.           self.boundary_left = boundary_left
336.           self.boundary_right = boundary_right
337.           self.boundary_top = boundary_top
338.           self.boundary_bottom = boundary_bottom
339.           self._prepare_boundary_values()
340.           self.matrix_F_origin = self.matrix_F.copy()
341.           self._adjust_F_for_boundary()
342.
343.       def _prepare_boundary_values(self):
344.           """ 根据边界条件计算非齐次方程 """
345.           x_points_total = np.linspace(0, self.x_max, self.n+2)
346.           y_points_total = np.linspace(0, self.y_max, self.n+2)
347.           self.boundary_values = np.zeros((self.n+2, self.n+2))
348.           self.boundary_values[:, 0] = self.boundary_left(y_points_total)
349.           self.boundary_values[:, -1] = self.boundary_right(y_points_total)
350.           self.boundary_values[0, :] = self.boundary_bottom(x_points_total)
351.           self.boundary_values[-1, :] = self.boundary_top(x_points_total)
352.
353.       def _adjust_F_for_boundary(self):
354.           """ 根据非齐次边界条件调整 F 矩阵。调整之后不必重写 LU 分解与 DST 计算方法
    了。 """
355.           h2 = self.x_step**2
356.           k2 = self.y_step**2
357.           self.matrix_F[:, 0] -= self.boundary_values[1:-1, 0] / k2
358.           self.matrix_F[:, -1] -= self.boundary_values[1:-1, -1] / k2
359.           self.matrix_F[0, :] -= self.boundary_values[0, 1:-1] / h2
360.           self.matrix_F[-1, :] -= self.boundary_values[-1, 1:-1] / h2
361.           print(self.matrix_F)
```

```
362.
363.     def method_jacobi_iteration_convolve(self, max_iter=20000, tol=1e-4):
364.         """ 卷积加速的 Jacobi 迭代法，适应非齐次边界条件 """
365.         factor = 1 / (2/self.x_step**2 + 2/self.y_step**2)
366.         kernel = np.array([
367.             [0, 1/self.y_step**2, 0],
368.             [1/self.x_step**2, 0, 1/self.x_step**2],
369.             [0, 1/self.y_step**2, 0],
370.             ]) * factor
371.         iter_count = 0
372.
373.         U_new = np.zeros((self.n,self.n))
374.         U_old = np.zeros((self.n,self.n))
375.
376.         while iter_count < max_iter:
377.             # 应用边界条件
378.             U_with_boundary = np.pad(U_old, pad_width=1, mode='constant', co
    nstant_values=0)
379.             U_with_boundary[1:-1, 0] = self.boundary_values[1:-
    1, 0]    # Left boundary
380.             U_with_boundary[1:-1, -1] = self.boundary_values[1:-1, -
    1]  # Right boundary
381.             U_with_boundary[0, 1:-1] = self.boundary_values[0, 1:-
    1]    # Top boundary
382.             U_with_boundary[-1, 1:-1] = self.boundary_values[-1, 1:-
    1]  # Bottom boundary
383.
384.             # 卷积运算
385.             U_new = convolve2d(U_with_boundary, kernel, mode='valid') - self
    .matrix_F_origin * factor
386.
387.             diff = np.max(np.abs(U_old - U_new))
388.             if diff < tol:
389.                 break
390.             U_old = U_new.copy()
391.             iter_count += 1
392.         else:
393.             print(f"注意：已经超过最大迭代数{max_iter}，结果可能不收敛。")
394.         return U_new
395.
396.     def visualize(self, U):
397.         # 生成网格点，这次包括边界
398.         X, Y = np.meshgrid(
399.             [0] + list(self.x_points) + [self.x_max],
```

```
            [0] + list(self.y_points) + [self.y_max]
        )

        # 为解 U 添加边界值
        U_with_boundary = np.pad(
            U, pad_width=1, mode='constant', constant_values=0
        )
        U_with_boundary += self.boundary_values

        # 创建 3D 图形
        fig = plt.figure()
        ax = fig.add_subplot(111, projection='3d')

        # 绘制曲面图
        surf = ax.plot_surface(X, Y, U_with_boundary.T, cmap='Blues')

        # 添加颜色条以表示高度值
        fig.colorbar(surf, shrink=0.5, aspect=5)

        # 设置标题
        ax.set_title('Numerical Solution with Nonhomogeneous Boundary Conditions')

        # 设置图形的长宽比为相等，这样 x, y, z 轴的一个单位将被显示为相同的长度
        ax.set_aspect('auto')

        # 保存图像
        plt.savefig("result_with_nonhomogeneous_boundary.png", dpi=800)

        # 显示图像
        plt.show()



def main():
    lattice_function = lambda x, y: -2 * np.pi**2 * np.sin(np.pi * x) * np.sin(np.pi * y)
    analytical_solution = lambda x, y: np.sin(np.pi * x) * np.sin(np.pi * y)

    time_error_analysis(lattice_function, 1, 1, analytical_solution, range(10,120,3))
```

```
440.
441.       test = NumericalSolutionForPossionEquation(lattice_function,1.5,1.5)
442.       U = test.method_dst()
443.       test.visualize(U)
444.
445.       def lattice_function(x, y):
446.           return 4 * np.ones_like(x) * np.ones_like(y)
447.
448.       def boundary_left(y):
449.           return y**2
450.
451.       def boundary_right(y):
452.           return 1 + y**2
453.
454.       def boundary_bottom(x):
455.           return x**2
456.
457.       def boundary_top(x):
458.           return x**2 + 1
459.
460.       def analytical_solution(x, y):
461.           return x**2 + y**2
462.       test = NonhomogeneousPoissonEquation(lattice_function, boundary_left, bo
    undary_right, boundary_top, boundary_bottom, n=40)
463.       U = test.method_dst()
464.       print(test.check_analytical_solution(U, analytical_solution))
465.       test.visualize(U)
```